

Feature: Contact and booking

From build-plan: feature 7

Branch: `feature/contact-and-booking`

Goal

`/contact` , `/thank-you` and their `/es` twins: the page that turns a visitor into a booking.

The project plan calls this form load-bearing, and it means two separate things by that. It is **the booking route for anyone who will not DM** - the overview's second audience, a mother paying for a grad set who will not message a teenager on Instagram. And it is **the only attribution the site has**: no analytics, no cookie banner, so the "how did you find me" field is the entire measurement apparatus for whether any of this worked.

Alongside it sits the direct route for everyone who *will* DM.

In scope

- One zod schema, shared by `react-hook-form` on the client and the Server Action on the server. Validated on both sides.
- The `/contact` route in both locales: the form, and the direct booking route beside it.
- The "how did you find me" field, as a select with fixed options.
- A Server Action that sends the enquiry through Resend and never pretends to have succeeded.
- A honeypot, because a public form on a public site gets bots.
- `/thank-you` in both locales, excluded from search.
- `ContactPage` JSON-LD, metadata, canonical and hreflang for both routes.

Out of scope

- **Real booking.** Item 11, post-MVP. No availability, no deposits, no calendar. This feature sends an email.
- **Storing submissions.** The overview is explicit: transient, validated, emailed, never retained. No database, no log of enquiry contents.
- **Prefilling the service from a query parameter.** A `/contact?service=nails` link from the service pages would be convenient and would make the page dynamic rather than static. Out for now; the select defaults to none.
- **Rate limiting.** There is no store to count against. The honeypot is the only abuse control, and that is a deliberate first-pass choice.

- **A privacy policy or cookie banner.** The plan says neither is needed at this scale and nothing here changes that: nothing is stored and nothing is tracked.
- **Sitemap and robots.txt.** Item 9, though `/thank-you` sets its own noindex here rather than waiting.
- **~~Any new package.~~ Reversed during implementation, with approval.** Two were added so the enquiry email could be HTML rather than plain text: `@react-email/components` and `@react-email/render`. See build step 7.

Build loop

`blueprint/config.json` sets `workflow.stepReview: "feature"` and `workflow.checkpointCommits: "disabled"`: steps in order, no commits during implementation, one review at the end. `/complete` makes the single commit.

No declared Verify command. The gate is `npm run build` (TypeScript runs inside it) and `npm run lint`, plus exercising the form. Baseline on `main` at spec time: build passes, eleven static routes.

Step 4 cannot deliver mail. There is no Resend key, no verified sender and no destination address - see Open questions. The action, its guard and every failure path are still buildable and testable; only an actually-sent email is not. Steps 1, 2, 3, 5 and 6 are unaffected.

Build steps

- ☒ **1. The schema and the source options.** `lib/contactValidation.ts` exporting the zod schema and its inferred type, and `data/contactSources.ts` listing the "how did you find me" keys. Field constraints in Data / contracts. **Done when:** `npm run build` passes; the schema rejects an empty name, a malformed email, a message under the minimum and a `source` outside the allowed keys; and removing a Spanish source label fails the build.
- ☒ **2. The contact route, page head and the direct route.** `app/[locale]/contact/page.tsx`: eyebrow, H1, lede, its own `alternates`, and the direct booking route beside the form slot. No form yet. **Done when:** `/contact` and `/es/contact` prerender with their own H1 and canonical; `/en/contact` 301s to `/contact`; and the direct route renders as a link when `siteConfig.social.instagram` is set and as prose when it is not, exactly as `BookingBand` and the footer already behave.
- ☒ **3. The form.** `components/contact/ContactForm.tsx`, `"use client"`, wired with react-hook-form directly - **not** shadcn's `Form` wrapper, per the coding standards. Every state in Data / contracts, including the accessibility contract. **Done when:** submitting empty shows one message per invalid field, each associated with its input and announced; focus moves to the first invalid field; the submit button reports a busy state and cannot double-submit; a server failure shows a message that names the alternative route rather than an error code; and the form works at 375px.
- ☒ **4. The Server Action and Resend delivery.** *Delivery proved locally.* `actions/contactAction.ts`, `"use server"`. Re-validates with the same schema, short-circuits the honeypot, sends through

Resend, returns the result shape. **Done when:** the action re-validates rather than trusting the client; a filled honeypot returns success without sending; a missing `RESEND_API_KEY` returns the failure result and logs server-side rather than throwing or silently succeeding; a Resend error response is treated as a failure; and no submission value is ever written to disk.

- ☒ **5. The thank-you page.** `app/[locale]/thank-you/page.tsx`, reached after a successful submit. **Done when:** both locales prerender; each emits `robots: { index: false, follow: true }`; the page states what happens next and offers a way back into the site; and a successful submit lands on the locale-correct one.
- ☒ **6. ContactPage JSON-LD and the closing pass.** `contactPageSchema` in `lib/schema.ts`. Re-read both pages against "nothing ships with placeholder text". **Done when:** `/contact` emits one `ContactPage` block naming the business from `siteConfig`; Calgary appears in the title, H1 and copy; no sentence is shared with another page; and `npm run build` and `npm run lint` pass clean.
- ☒ **7. The HTML enquiry email.** *Added after review, with approval.* `emails/contact-enquiry.tsx` rendered through Resend's `react` option, with the plain-text part kept as the fallback. **Done when:** a real submission arrives as the branded HTML version rather than a wall of text, the message keeps the line breaks the visitor typed, and `npm run build` and `npm run lint` pass clean.

Files / areas

New:

- `app/[locale]/contact/page.tsx`
- `app/[locale]/thank-you/page.tsx`
- `components/contact/ContactForm.tsx` - "use client"
- `actions/contactAction.ts` - "use server"
- `lib/contactValidation.ts` - the one schema
- `data/contactSources.ts`

Changed:

- `dictionaries/en.ts`, `dictionaries/es.ts` - `contact`, `thankYou` blocks
- `lib/schema.ts` - `contactPageSchema`
- `data/siteConfig.ts` - `business.email`, the destination. See Open questions
- `.env.local` - `RESEND_API_KEY`, `RESEND_FROM`. Untracked

Unchanged: `lib/locale.ts` already lists `/contact` and `/thank-you` in `routes`, so the rewrites and the `/en` redirect exist. The header CTA, the footer and every `BookingBand` already point at `/contact`.

Data / contracts

The schema, in `lib/contactValidation.ts`, and the single source of truth for both sides. The client validates for the visitor's benefit; the server validates because the client cannot be trusted.

| Field | Rule | Why | |---|---|---| | `name` | required, trimmed, 1-80 | | `email` | required, valid address, ≤ 120 | The only way she can reply | | `service` | `ServiceId` or `null` | Optional; "not sure yet" is a real answer | | `source` | required, one of the fixed keys | The only attribution the site has | | `message` | required, trimmed, 10-2000 | 10 stops "hi"; 2000 stops a paste bomb | | `company` | must be empty | Honeytrap. Never rendered to a sighted user |

source is a select, not free text. The plan says this field is how success gets measured, and text answers cannot be counted - "insta", "Instagram", "my cousin showed me" are three rows nobody can add up. Fixed keys in `data/contactSources.ts` with locale-keyed labels, following the `faq.ts` pattern so a missing Spanish label is a build error. Required, with a final `other` option so nobody is trapped.

The result shape, matching the coding standards' `{ success, error }`:

```
export type ContactResult =  
  | { success: true }  
  | { success: false; error: string };
```

`error` is a key the form turns into a sentence, never a raw error and never a Resend code. The standards are explicit: a failed contact form must never look like a success, and the visitor gets the friendly version while the server keeps the useful one.

The honeypot returns `success: true` without sending. A bot that learns which field betrayed it comes back without it.

The email is HTML with a plain-text part. Corrected during implementation. This spec first ruled out a React email component because it would need a package this project did not have. Asked, and approved: both `@react-email/components` and `@react-email/render` were added.

The plain-text part stays regardless. It is what a text-only client renders and what a spam filter reads when it distrusts the HTML, and it costs one array.

React escapes every interpolation, so a visitor who types markup into the message field gets it back as text. Nothing a visitor types is ever rendered on the site itself, so there is no XSS surface on the web side either.

The email carries the locale. A visitor who used `/es/contact` should be answered in Spanish, and nothing else in the message would tell her which language to reply in.

replyTo is the visitor's address. `from` is a send-only address at a verified domain with no mailbox behind it, so a reply to it vanishes. `email` is required here, so unlike the sibling project there is no fallback case.

Field-level rendering and announcement, which the form must satisfy:

- every input has a `<label>` associated by `htmlFor / id`, not a placeholder standing in for one
- an invalid field sets `aria-invalid` and points `aria-describedby` at its error
- the error text is in a live region so it is announced, not just painted
- on a failed submit, focus moves to the first invalid field

- the submit control reports busy and cannot fire twice
- the form-level failure message names the direct route as the way through

Metadata. Both pages set their own `alternates`, for the reason every page since the gallery has needed: the layout points canonical and all three hreflang values at `/`, and a child's `alternates` replaces rather than merges.

`/thank-you` sets `robots: { index: false, follow: true }`. A thank-you page in the index ranks for nothing and can surface to someone who never submitted. `follow` stays on so the links out of it still pass.

`ContactPage` JSON-LD on `/contact` only, naming the business from `siteConfig` and `areaServed` Calgary, following the four helpers already in `lib/schema.ts`.

Testing

No unit tests: no runner is configured and `/tests` has not been run. **This is the first feature with logic worth testing** - a validation schema is exactly what a unit test is for - so if you want `/tests`, this is the feature that justifies it. Without it, the schema is exercised by hand.

No browser harness, so nothing here claims automated browser coverage. The form's states are the one thing on this site that genuinely cannot be checked from build output: a validation message, a focus move and a busy button are runtime behaviour. Expect to run the dev server for step 3.

Notes for the AI

Do not use shadcn's `Form` / `FormField` wrapper. The coding standards name this specifically: shadcn's form layer has not caught up with react-hook-form's current API, and mixing them produces a form that renders but does not work. Verified on 2026-09-09. Use shadcn's Input, Label, Select and Button, and wire react-hook-form directly.

A failed send must never look like a success. Silently swallowing a Resend failure loses a booking and nobody finds out. Resend reports failures in the response rather than by throwing, so both the returned `error` and a `try/catch` are needed - one alone misses half the cases.

Nothing is stored. No database, no file, no log of message contents. Log that a send failed and why Resend said so; never log the enquiry itself.

The secret stays server side. `RESEND_API_KEY` is read inside the action only. Anything a client component can reach is public whatever it is named.

No component contains the word Dani. The destination address, the business name and the Instagram handle all come from `siteConfig`.

Reuse. `Band`, `BandHead`, `Hot` and `BookingBand` are in `components/site/`. `components/ui/button.tsx` exists; Input, Label and Select do not yet and come from shadcn.

Open questions

1. Resend: answered for local, still blocked for production. Settled on 2026-09-22.

- **Destination:** `frankdmosquera@gmail.com` . Dani has no business address yet, so enquiries go to the agency inbox for now. This is a placeholder for a real one and belongs in `siteConfig.business.email` with that said out loud, so nobody ships it to a live site believing it is hers.
- **RESEND_API_KEY** is copied from the shared account that already serves `primo-painters` and `face-and-body` . One account, one key, three projects.
- **RESEND_FROM** is `onboarding@resend.dev` , and it is local-only. The account has exactly two verified domains, `primopainters.ca` and `faceandbodywellnesscentre.com` , and both belong to other clients. Sending a nail studio's enquiries from a painting company's domain is wrong even in development, so the shared sender is used instead. It delivers only to the address owning the Resend account and 403s every other recipient.

Still blocked for production, and the blocker is upstream of this feature: facing-dani has no domain, so there is nothing to verify in Resend. Do not set `RESEND_FROM` in Vercel until that changes - a deploy that can only mail one inbox is worse than one that fails loudly.

One thing to watch: `onboarding@resend.dev` only reaches the Resend account owner. If that account is not registered to `frankdmosquera@gmail.com` , every local send returns 403 - which the action surfaces as a visible failure rather than swallowing, so it will be obvious on the first test rather than silent.

Noticed while checking, and not this project's problem: `faceandbodywellnesscentre.com` reports `partially_failed` in Resend. That is `face-and-body` 's production sender, so its contact form may be degraded right now. Flagged rather than touched. **2. The Instagram handle is still null, so the direct booking route has no link.** `siteConfig.social.instagram` has been `null` since feature 1, and the footer and every `BookingBand` already handle it by rendering the link only when it is set. This page does the same, so it ships either way - but "the direct booking route" in the build-plan line is precisely that link, and until the handle lands, half of this feature's title is prose rather than a route.

3. Should `source` include a free-text escape? Spec'd as a fixed select with an `other` option and no text box, because the field exists to be counted. The cost is that "my cousin Maria sent me" arrives as "other" and the specific referral is lost. Adding an optional text input that appears when `other` is chosen is a small change and a reasonable one - say so now rather than after the keys are in both dictionaries.

Status: verified end to end, on a running server

`npm run build` exit 0, `npm run lint` exit 0. Fifteen routes prerender.

This is the first feature on this project proved against a running app rather than build output. The dev server was started on an assigned port, because port 3000 was occupied by `face-and-body` and taking

it would have stopped another project.

Observed in the browser:

| Check | Result | |---|---| | Submit empty | 4 fields `aria-invalid`, each `aria-describedby` an existing error node, 4 announced messages, focus moved to the first | | `service` on empty submit | correctly not flagged - it is optional | | Real send, English | landed on `/thank-you`, Resend accepted in 435ms | | Real send, Spanish | `/es/contact` to `/es/thank-you`, email carried `Language: es` | | `RESEND_API_KEY` unset | failure message shown, stayed on `/contact`, logged `[contact] RESEND_API_KEY is not set; nothing was sent` | | Honeypot filled, key still unset | success and redirect, action returned in **0ms**, nothing sent | | HTML email | rendered and delivered after both packages were declared |

Parsed out of the prerendered markup:

| | `/contact` | `/es/contact` | |---|---|---| | Form controls | 6, all with ids | 6 | | Every control labelled | yes | yes | | `noValidate` on the form | yes | yes | | Honeypot `tabindex="-1"` inside `hidden aria-hidden` | yes | yes | | Service options / source options | 3 / 5 | 3 / 5 | | Direct route | prose, handle unset | prose | | `ContactPage` JSON-LD | one | one |

`/thank-you` and `/es/thank-you` both emit `noindex, follow`.

Departures from the spec

1. **The email became HTML, reversing a documented decision.** The spec ruled out a React email component because it needed a package this project did not have. Asked during review, approved, and two packages were added: `@react-email/components` and `@react-email/render`. 2. **Two packages, not one.** `@react-email/components` alone is not enough - Resend throws `Failed to render React component. Make sure to install @react-email/render or @react-email/components`. The first HTML send failed on exactly that, which the failure path surfaced correctly rather than swallowing. 3. **Both selects are typed `string`, not unions of their allowed values.** The union version cost two build rounds: a zod union narrows differently on input and output, so the resolver's generics stop matching `useForm`'s. Runtime validation is unchanged - an unknown value is rejected by the refine and again on the server. The reasoning is written into `lib/contactValidation.ts`. 4. **The form controls are plain elements in the site's theme, not shadcn's Input, Label and Select.** The coding standards name those components, and this is a deliberate departure: `components/ui/button.tsx` is the only shadcn component in the project and nothing imports it, every other component here is a raw element with brand tokens, and shadcn's are styled for its own palette. A native `<select>` is also the better control on a phone - system picker, no JavaScript needed. **The part of that standard that matters was honoured exactly:** no shadcn `Form` / `FormField` wrapper. 5. **`.claude/launch.json` was added** so the dev server could be started on an assigned port. Not part of the feature's product surface, but it is what made the verification above possible, and it is useful to keep.

The bug the browser found

The honeypot was inverted, and the build was perfectly happy.

`company` was `z.string().max(0)` and the check sat after `safeParse`. A filled honeypot therefore failed validation, the action returned its failure result, and the form showed "that did not send". That is a tell: a bot that learns which field betrayed it comes back without the field, which is the exact outcome a honeypot exists to prevent.

Fixed by checking the honeypot **before** parsing and leaving `company` unconstrained. Proved by filling it with `RESEND_API_KEY` still unset: the page redirected to `/thank-you` anyway - indistinguishable from a real success - and the log shows the action returning in 0ms, having never reached Resend.

Nothing in a type system or a build would have caught this.

Noticed in a sibling project, not touched

Two problems in `face-and-body`, both found while setting this up and both reported rather than fixed:

- `faceandbodywellnesscentre.com` reports `partially_failed` in Resend. That is its production sender.
- It uses `@react-email/render` without declaring it. The package resolves only because it is hoisted from the `react-email` devDependency. An install with `--omit=dev` would drop it and every enquiry would fail at send, exactly as the first attempt failed here.

Not verified

- **The rendered email was never seen by this session.** It sends, and it is the React component rather than the text fallback, but whether it looks right in Gmail was judged by the person who received it. An attempt to render it to a local HTML file was abandoned: `@react-email/render` is not resolvable as a top-level module here and Node's type stripping would not load the `.tsx`.
- **Production delivery is impossible and must stay that way for now.** `RESEND_FROM` is Resend's shared sender, which reaches only the account owner. This project has no domain, so there is nothing to verify. Setting `RESEND_FROM` in Vercel before a real domain exists would produce a deploy that silently mails one inbox.
- `siteConfig.business.email` is the agency's address, not hers. Every booking the site produces lands in somebody else's inbox until she has one.
- **No rate limiting.** The honeypot is the only abuse control. There is no store to count against and that was a deliberate first-pass choice.
- **The Spanish copy is draft.** Dani has not read it.
- **A transient 500 on HEAD /** appeared once during dev-server cold start, before `/en` had finished compiling. Not reproducible - all routes return 200 for HEAD afterwards - and not in this project's code.

Implementation walkthrough

One schema, two callers

`lib/contactValidation.ts` is the whole trust boundary. `react-hook-form` validates against it in the browser so the visitor is told before she submits; `submitContact` validates against it again because a Server Action is a public endpoint and anything that can POST can reach it with any body at all.

Its error messages are **keys, not sentences** - `"required"`, `"email"`, `"tooShort"`, `"tooLong"` - because one schema serves both languages and a hard-coded "Required" would leak English onto the Spanish page. The form maps each key through the dictionary.

The one scar in the file is that both selects are typed `string` rather than unions. That is written up in a comment rather than left to be rediscovered: a zod union narrows differently in and out, which breaks the resolver's generics against `useForm`, and it cost two rounds to learn.

The action, and the order of its checks

`actions/contactAction.ts` does four things in a deliberate order: honeypot, validate, check the key, send.

The honeypot is **first** and its answer is success. That order was wrong once and produced the bug described above. A presence check on an arbitrary string needs no validation to be safe, and looking indistinguishable from a real success is the entire mechanism.

Everything past validation returns the same opaque `"failed"` to the visitor. A blocked API key and a rejected recipient are the same problem from where she is sitting, and the form's copy points her at the DM either way. The real reason goes to the server log - the split the coding standards ask for.

Resend reports failures in the response rather than by throwing, so both the returned `error` and a `try/catch` are needed. One alone misses half the cases, and a failed contact form that looks like a success loses a booking silently.

Nothing is stored. The log records that a send failed and what Resend said; it never records the enquiry.

The form

`components/contact/ContactForm.tsx` is the only client component in the feature. It is wired with `react-hook-form` directly, not with shadcn's `Form` wrapper, which the coding standards single out as producing a form that renders but does not work.

`noValidate` is on the form so our messages run instead of the browser's, which means the accessibility contract is ours to keep: every input has a real `<label>` rather than a placeholder standing in for one, an invalid field sets `aria-invalid` and points `aria-describedby` at its error, errors are in live regions, and a failed submit moves focus to the first invalid field. That last one matters most - without it a keyboard user is left at the submit button with the errors somewhere above them.

The honeypot sits in a `hidden aria-hidden` wrapper with `tabIndex=-1`, so a human never reaches it by eye, mouse, keyboard or screen reader, and something filling every field in the DOM does.

Why the controls are not shadcn's

Every other component on this site is a raw element with brand tokens. `button.tsx` is the only shadcn component present and nothing imports it. Its styles are written against shadcn's own palette, so the form controls would have needed restyling to the After Party theme anyway, and adding packages needs asking. A native `<select>` is additionally the right control on a phone: it opens the system picker and works with no JavaScript at all.

The email

`emails/contact-enquiry.tsx` exists because plain text was not good enough to read on a phone. Its brand colours are inlined as literal hex rather than read from the theme, and that duplication is deliberate: email clients do not load a stylesheet and do not support CSS custom properties, so a token would render as nothing.

The plain-text part was kept. It is what a text-only client renders and what a spam filter reads when it distrusts the HTML, and it costs one array.

`createElement` rather than JSX keeps the action a `.ts` file. One call does not justify renaming the module and churning every import of it.

The thank-you page

A real route rather than a message swapped into the form, because a URL change is the clearest possible signal that something happened - it survives a back button, a refresh and a screen reader. It sets `noindex, follow`: it ranks for nothing useful, and arriving on it from a search result would tell someone their message was sent when it was not.